MATLAB ONRAMP - SECTION THREE

Quantization and SNR

Sampling, quantizing and measuring signal quality in MATLAB

Under the supervision of **DR. Lobna Abouelmagd**

Prepared by **ENG. Moamen Yasser**

MATLAB ONRAMP - SECTION THREE

Quantization and SNR

Part 1: Example 2.13

Quantize a sine wave · Part 1 of 2

THE PROBLEM

Example 2.13: quantize a sine wave

A sinusoid $x(t) = 4.5 \cdot \sin(2\pi \cdot 100t)$ is sampled at 8,000 Hz.

a) Quantize it with 4 bits for a range of −5 V to 5 V and plot it.

b) Calculate the SNR due to quantization.

100 Hz
signal frequency

8,000 Hz
sampling rate

4 bits
16 steps

±5 V
signal range

goal: turn a smooth sine into a staircase of levels, then measure how good it is

TASK 1

Set the sampling parameters

Name the signal frequency **signalFrequency**, the sampling rate **samplingRate** and the sampling period **samplingPeriod = 1/samplingRate**. The time vector **timeVector** steps by samplingPeriod from 0 to 0.02 s, giving 161 samples.

```
signalFrequency=100;
samplingRate=8000;
samplingPeriod=1/samplingRate;
timeVector=0:samplingPeriod:.02;
```

create signalFrequency, samplingRate, samplingPeriod and a time vector timeVector from 0 to 0.02 s

TASK 2

Generate the original signal

Apply the sine formula to every element of **timeVector** at once. **originalSignal** is a vector of 161 samples with a peak of 4.5 V.

```
%original signal

originalSignal= 4.5*sin(2*pi*100*timeVector);
```

build originalSignal with originalSignal = 4.5*sin(2*pi*100*timeVector) and check that max(originalSignal) is close to 4.5

TASK 3

Define the quantization rule

A 4-bit quantizer has **numLevels = 2^4 = 16** steps across the range. The step size is **stepSize = (maxVoltage - minVoltage)/numLevels = 10/16 = 0.625 V**.

```
%quantization rule

maxVoltage=5;

minVoltage=-5;

numLevels=2^4;

stepSize=(maxVoltage-minVoltage)/numLevels;
```

set maxVoltage, minVoltage and numLevels, then compute stepSize and check that it equals 0.625

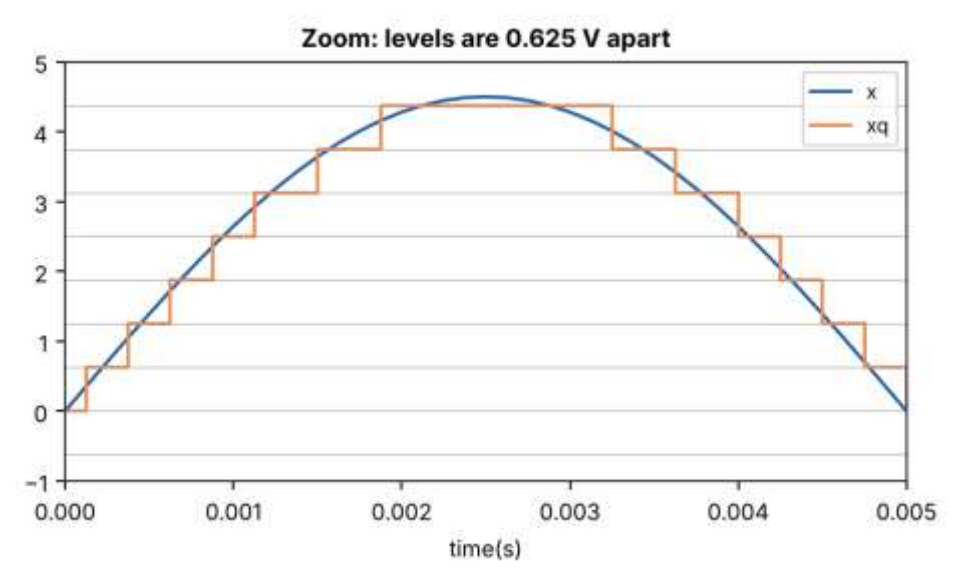
TASK 4

Quantize the signal

Shift originalSignal so the range starts at 0, divide by **stepSize** and round to a whole step **levelIndex**. Convert back to volts with **quantizedSignal**.

```
levelIndex=round((originalSignal-minVoltage)/stepSize);

quantizedSignal=minVoltage+levelIndex*stepSize;
```



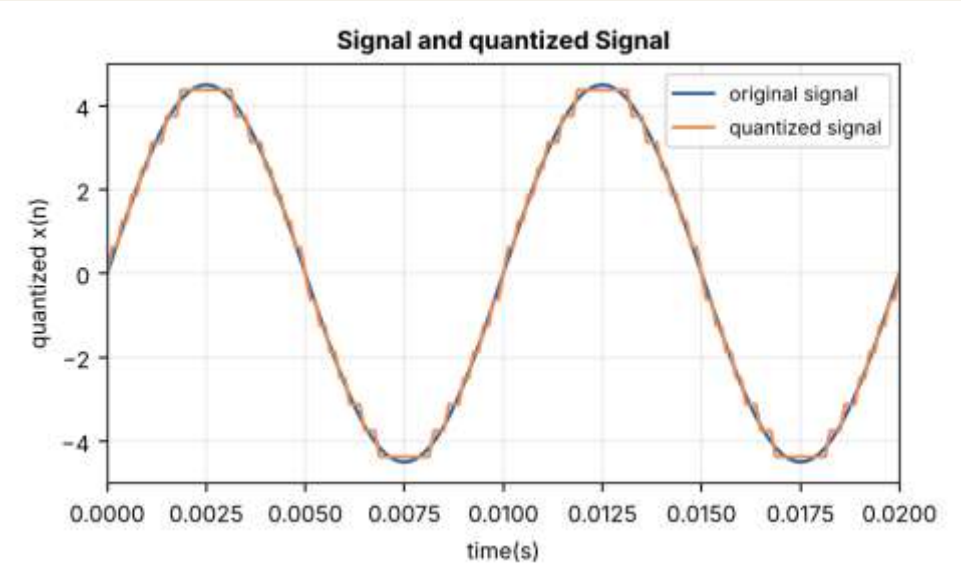
compute levelIndex and quantizedSignal, then check that every quantizedSignal value sits on a multiple of 0.625 above -5

TASK 5

Plot original and quantized

Plot both signals on the same axes, then add a legend, a title and axis labels so the figure can be read.

```
plot(timeVector,originalSignal,timeVector,quantizedSignal);
legend('original signal','quantized signal');
title('Signal and quantized Signal');
xlabel('time(s)');
ylabel('quantized x(n)');
```



plot originalSignal and quantizedSignal together and add the legend, title and labels from the code

TASK 6

Measure the quantization error

The error is the quantized value minus the original value. **`abs(quantizationError)`** gives its size, which can never exceed **stepSize/2 = 0.3125 V**.

```
quantizationError=quantizedSignal-originalSignal;

abs(quantizationError);
```

compute quantizationError and check that max(abs(quantizationError)) is below 0.3125

FULL EXAMPLE

Example 2.13 part a: the full script

```
signalFrequency=100;
samplingRate=8000;
samplingPeriod=1/samplingRate;
timeVector=0:samplingPeriod:.02;
%original signal
originalSignal= 4.5*sin(2*pi*100*timeVector);
%quantization rule
maxVoltage=5;
minVoltage=-5;
numLevels=2^4;
```

```
stepSize=(maxVoltage-minVoltage)/numLevels;
levelIndex=round((originalSignal-minVoltage)/stepSize);
quantizedSignal=minVoltage+levelIndex*stepSize;
plot(timeVector,originalSignal,timeVector,quantizedSignal);
legend('original signal','quantized signal');
title('Signal and quantized Signal');
xlabel('time(s)');
ylabel('quantized x(n)');
quantizationError=quantizedSignal-originalSignal;
abs(quantizationError);
```

TASK 7

Part b: signal and error powers

The mean of the squared samples is the average power. Find it for the signal (**signalPower**) and the error (**errorPower**). The `.^` operator squares every element.

```
signalPower = mean(originalSignal.^2);

errorPower = mean(quantizationError.^2);

snrRatio = signalPower / errorPower;
```

compute signalPower, errorPower and snrRatio, and check that SNR is about 378

TASK 8

Convert to dB and print

Engineers quote SNR in decibels: **snrDb = 10·log10(snrRatio)**. In **`fprintf`**, **`%.2f`** prints a number with 2 decimals and **`\n`** starts a new line.

```
snrDb = 10 * log10(snrRatio);
fprintf('The SNR due to quantization is: %.2f dB\n', snrDb);
```

Command Window
```
The SNR due to quantization is: 25.78 dB
```

run the script and check that the Command Window prints 25.78 dB

MATLAB ONRAMP - SECTION THREE

Quantization and SNR

Part 2: Example 2.14

Quantize a signal stored in a file · Part 2 of 2

THE PROBLEM

Example 2.14: a signal from a file

A speech signal sampled at 8,000 Hz is stored in a file.

a) Quantize it with a 4-bit quantizer for −5 V to 5 V.

b) Plot the original, quantized and error signals.

c) Calculate the SNR.

8,000 Hz
sampling rate

4 bits
16 steps

±5 V
signal range

3 plots
signal, quantized, error

goal: save samples to a file, load them back, quantize and compare three plots

TASK 1

Write the samples to a file

No data file? Create one. **`fopen`** opens signal.txt for writing (**`'w'`**), **`fprintf`** writes each sample on its own line and **`fclose`** closes the file.

```
signalFrequency=100;
samplingRate=8000;
samplingPeriod=1/samplingRate;
timeVector=0:samplingPeriod:.25;
originalSignal= 4.5*sin(2*pi*100*timeVector);
fileId=fopen('signal.txt','w');
fprintf(fileId,'%f\n',originalSignal);
fclose(fileId);
```

create signal.txt, then open it in the Current Folder and check that it holds 2001 lines

TASK 2

Load the file back

load reads the numbers in signal.txt into the vector **speechSignal**. The file holds only values, so rebuild the time axis from the number of samples and **samplingRate**.

```
samplingRate=8000;

speechSignal=load('signal.txt');

timeVector=(0:length(speechSignal)-1)/samplingRate;
```

load signal.txt into speechSignal and check that length(speechSignal) is 2001

TASK 3

Quantize the loaded signal

The same rule as before, with **speechSignal** in place of originalSignal. Only the input name changes.

```
maxVoltage=5;

minVoltage=-5;

numLevels=2^4;

stepSize=(maxVoltage-minVoltage)/numLevels;

levelIndex=round((speechSignal-minVoltage)/stepSize);

quantizedSignal=minVoltage+levelIndex*stepSize;
```

quantize speechSignal and check that quantizedSignal has 2001 values

TASK 4

Plot with subplot and stairs

`subplot(3,1,k)` makes a grid of 3 rows and 1 column and selects row k. **`plot`** draws the original; **`stairs`** draws the quantized signal as steps.

```
subplot(3,1,1)
plot(timeVector,speechSignal);
xlabel('time(s)');
ylabel(' original Speech');
subplot(3,1,2)
stairs(timeVector,quantizedSignal);
xlabel('time(s)');
ylabel(' Quantized Speech');
```

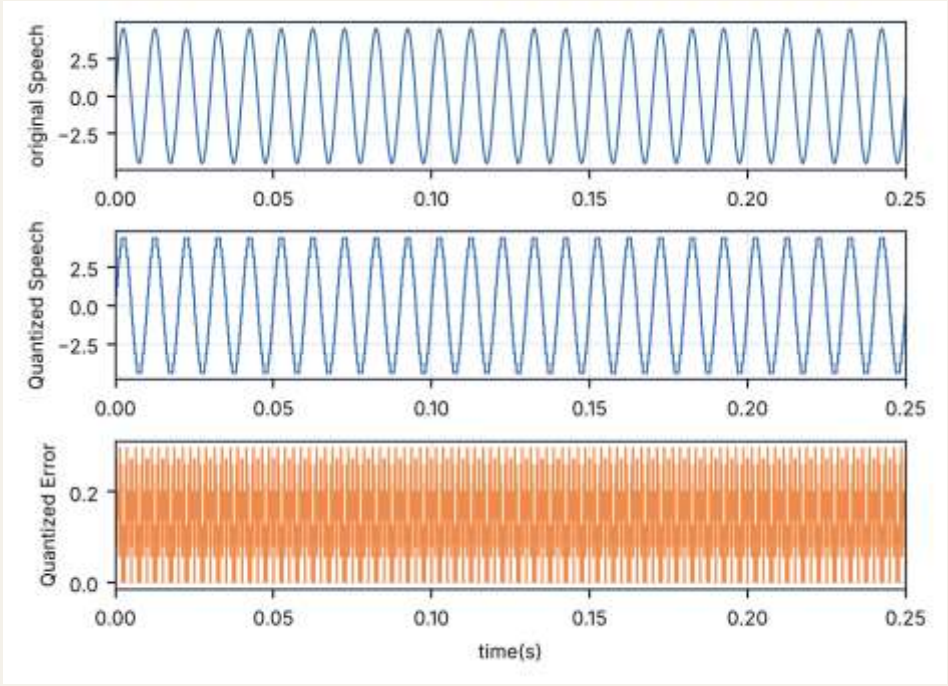
plot the original in row 1 and the quantized signal in row 2

TASK 5

Plot the quantization error

Take the absolute error and draw it in the third row to finish the figure.

```
quantizationError=abs(speechSignal-quantizedSignal);
subplot(3,1,3)
stairs(timeVector,quantizationError);
xlabel('time(s)');
ylabel(' Quantized Error');
```



add the error plot in row 3 and check that it never goes above 0.3125

FULL EXAMPLE

Example 2.14: the full script

```
samplingRate=8000;
speechSignal=load('signal.txt');
timeVector=(0:length(speechSignal)-1)/samplingRate;
maxVoltage=5;
minVoltage=-5;
numLevels=2^4;
stepSize=(maxVoltage-minVoltage)/numLevels;
levelIndex=round((speechSignal-minVoltage)/stepSize);
quantizedSignal=minVoltage+levelIndex*stepSize;
```

```
subplot(3,1,1)
plot(timeVector,speechSignal);
xlabel('time(s)');
ylabel(' original Speech');
subplot(3,1,2)
stairs(timeVector,quantizedSignal);
xlabel('time(s)');
ylabel(' Quantized Speech');
quantizationError=abs(speechSignal-quantizedSignal);
subplot(3,1,3)
stairs(timeVector,quantizationError);
xlabel('time(s)');
ylabel(' Quantized Error');
```

TASK 6

Part c: SNR from the file data

The same power ratio as in Example 2.13, now computed from **speechSignal** and the absolute error **quantizationError**, then converted to decibels.

```
signalPower = mean(speechSignal.^2);

errorPower = mean(quantizationError.^2);

snrRatio = signalPower / errorPower;

snrDb = 10 * log10(snrRatio);
```

compute signalPower, errorPower, snrRatio and snrDb, and check that snrDb is about 25.78

TASK 7

Print the result

Use **`fprintf`** to report the SNR. The result matches Example 2.13 because both programs quantize the same 100 Hz sine with 4 bits.

```
fprintf('The SNR due to quantization is: %.2f dB\n', snrDb);
```

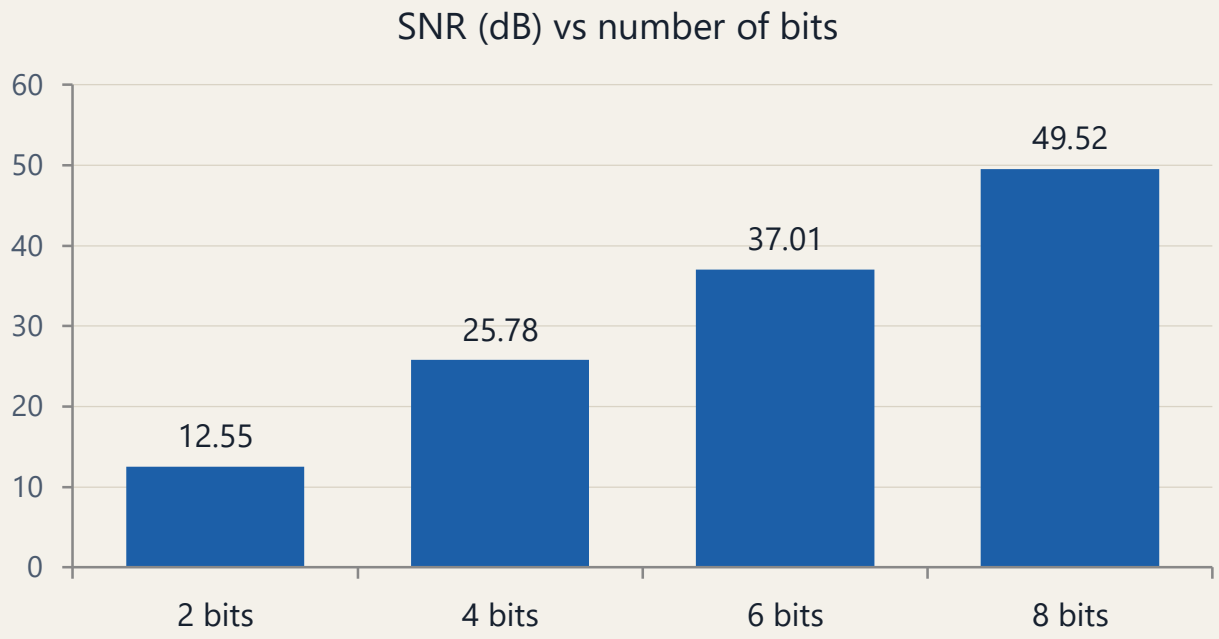
Command Window `The SNR due to quantization is: 25.78 dB`

run the whole script and check that the Command Window prints 25.78 dB

FURTHER PRACTICE

More bits, less error

Change **`numLevels=2^4;`** to **`numLevels=2^8;`** and run again. Each extra bit adds about **6 dB** of SNR.



set numLevels=2^8, rerun the script and check that the SNR is about 49.52 dB

FURTHER PRACTICE

Quick reference: functions used

`round(value)`	rounds each value to the nearest integer
`log10(values)`	base-10 logarithm, used for decibels
`fopen / fclose`	open and close a file ('w' = write)
`subplot(rows,cols,position)`	selects plot position of a rows-by-cols grid
`mean(values)`	average of the elements of values
`fprintf`	prints formatted text and numbers
`load('f.txt')`	reads numbers from a text file
`stairs(timeVector,values)`	draws a staircase (sample-and-hold)

MATLAB ONRAMP - SECTION THREE

Thank You

Any questions?

Under the supervision of **DR. Lobna Abouelmagd**

Prepared by **ENG. Moamen Yasser**